

SAS VIYA

Guide Expert — Compétences & Fonctionnalités

Fiche de référence expert — Prijanth Seevaratnam

Mai 2026

1. Vue d'ensemble

SAS Viya est la plateforme analytique cloud-native de SAS Institute. Elle modernise l'écosystème SAS traditionnel avec une architecture microservices, un moteur in-memory distribué (CAS), et des interfaces modernes pour l'analyse, la data science et l'IA. SAS Viya 4 (version actuelle) est déployable sur Kubernetes (OpenShift, GKE, AKS, EKS).

Positionnement marché

- Leader historique en analytique entreprise (secteurs : banque, assurance, gouvernement, santé)
- Concurrent direct : Dataiku, Azure ML, Databricks, AWS Sagemaker, Palantir
- Force principale : SAS language mûr, conformité réglementaire, richesse statistique
- Transition SAS 9.4 ! Viya : migration des bases installées vers le cloud
- Modèle de licence : SAS Cloud (SaaS), on-premise Kubernetes, partenariat AWS/Azure/GCP

Cas d'usage principaux

- Reporting réglementaire (IFRS 9, BCBS 239, ESG/Taxonomie, Pilier 3)
- Scoring de risque crédit, assurance, fraude
- Préviation et optimisation (time series, supply chain)
- Segmentation client et analyse comportementale
- Gouvernance des modèles ML en production (Model Manager)
- Décisionnel automatisé (Intelligent Decisioning)
- Qualité et gestion des données maîtres (MDM)

2. Architecture technique SAS Viya 4

Vue d'ensemble de l'architecture

SAS Viya 4 repose sur une architecture microservices déployée sur Kubernetes. Chaque composant est un service indépendant qui communique via API REST. Le moteur de calcul central est CAS (Cloud Analytic Services).

Couches principales

- Interface utilisateur : SAS Studio, SAS Drive, SAS Visual Analytics, SAS Model Manager
- API Layer : SAS Viya REST APIs (OpenAPI 3.0) — orchestration de tous les services
- Compute Layer : SAS Compute Server (sessions SAS classiques) + CAS (in-memory distribué)
- Data Layer : connexions aux sources (JDBC, ODBC, Cloud, HDFS, REST APIs)
- Infrastructure : Kubernetes (pods, services, ingress) + stockage persistant

CAS — Cloud Analytic Services

CAS est le moteur de calcul in-memory distribué de SAS Viya. C'est son composant le plus stratégique.

Architecture CAS

- Architecture SMP (Symmetric MultiProcessing) ou MPP (Massively Parallel Processing)
- CAS Controller : nœud maître qui orchestre les workers
- CAS Workers : nœuds de calcul qui stockent et traitent les données en mémoire
- Tables CAS : données chargées en RAM, distribuées sur les workers (partitionnées par blocs)
- Sessions CAS : contexte de connexion pour chaque utilisateur ou application

Caractéristiques CAS

- In-memory : toutes les tables actives sont en RAM !' latence très faible
- Distribué : les tables sont découpées en blocks répartis sur les workers
- Columnaire : stockage orienté colonnes pour une compression et une lecture efficace
- ACID partiel : transactions sur les tables CAS (append, update, delete)
- Persistance : tables en mémoire (volatiles) vs tables sauvegardées sur CASLIB
- Promotion : tables promues partagées entre toutes les sessions

SAS Compute Server

- Moteur SAS traditionnel (SAS 9 compatible) pour les programmes DATA step et PROC
- Exécution dans des sessions isolées (pods Kubernetes éphémères)
- Accès aux CASLIBS et aux librairies SAS classiques
- Connexion native à CAS : PROC CAS, libname CAS
- Interopérabilité avec le code SAS existant (SAS 9.4 !' Viya sans réécriture)

3. SAS Studio

Interface de développement

SAS Studio est l'IDE web de SAS Viya. Il remplace SAS Enterprise Guide pour le développement code et propose également une interface visuelle (Flows).

Fonctionnalités principales

- Éditeur de code SAS avec autocomplétion, coloration syntaxique, formatage
- Explorer : navigation dans les bibliothèques, tables, fichiers
- Results pane : visualisation des résultats log + output
- Tasks : assistants point-and-click pour les procédures statistiques courantes
- Snippets : bibliothèque de bouts de code réutilisables
- Shared content : dossiers partagés entre utilisateurs
- Git integration : versioning du code SAS depuis l'interface

Flows (Flux de données visuels)

- Interface drag-and-drop similaire au Flow de Dataiku
- Nœuds disponibles : Input data, Transformation, Output data, ML, Query
- Transformation visuelle sans code (équivalent Prepare de Dataiku)
- Connexion à des actions CAS pour les traitements distribués
- Scheduling via SAS Job Execution Service
- Export en code SAS généré automatiquement

Langage SAS — Fondamentaux

DATA Step

- Lecture/écriture de datasets SAS, transformations ligne par ligne
- Opérations : INPUT, OUTPUT, SET, MERGE, UPDATE, MODIFY
- Fonctions : caractère (substr, index, upcase), numérique (round, int, abs), date (datepart, intnx)
- Arrays : traitement de groupes de variables
- Boucles DO/END, conditions IF/THEN/ELSE, SELECT/WHEN
- Hash objects : jointures ultra-rapides en mémoire
- BY processing : traitement par groupe après tri

PROC SQL

- SQL complet avec extensions SAS (PROC SQL)
- Macro variables dans SQL : %let var = ...; SELECT &var. FROM...
- Création de tables : CREATE TABLE AS SELECT
- Jointures, sous-requêtes, fonctions fenêtre (PROC SQL 9.4+)
- CONNECT TO : SQL pass-through vers les bases externes
- REFLECT : récupération du schéma d'une table externe

Macro Language

- Génération dynamique de code SAS : %macro / %mend
- Variables macro : %let, &var., &var
- Conditions macro : %if %then %do / %else %do
- Boucles macro : %do i = 1 %to N
- Appel de macro : %nom_macro(param1, param2)
- Macros autocall : bibliothèques de macros partagées

Fonctions %sysfunc, %scan, %substr, %upcase, %eval

- %include : inclusion de fichiers SAS externes

4. SAS Visual Analytics

Présentation

SAS Visual Analytics (VA) est l'outil de reporting et d'exploration de données de SAS Viya. Il permet de créer des rapports interactifs, des tableaux de bord et des visualisations avancées sans code.

Fonctionnalités clés

- Exploration auto : analyse descriptive automatique d'une table CAS
- Rapports interactifs : drag-and-drop d'objets visuels sur un canevas
- Objets visuels : graphiques (bar, line, scatter, heatmap, treemap, geo, boxplot, waterfall...)
- Filtres dynamiques : contrôles (slider, listbox, text input) liés aux visuels
- Actions : drill-down, filter, link between reports, go to URL
- Données calculées : mesures agrégées, colonnes calculées, hiérarchies
- Paramètres : variables dynamiques pilotant les requêtes
- Mobile : rapports responsives pour smartphones/tablettes
- Export : PDF, PowerPoint, Excel, PNG
- Scheduling : envoi automatique par email à intervalle régulier
- Commenting & Annotations : collaboration sur les rapports

Sources de données

- Tables CAS (principal) : données in-memory haute performance
- SAS datasets sur CASLIB : chargement automatique dans CAS
- Sources JDBC/ODBC : connexion directe sans chargement CAS
- Google Analytics, Salesforce, etc. via connecteurs
- Données géospatiales : shapefiles, lat/lon, geo-encoding

Bonnes pratiques VA

- Pré-agrégé les données dans CAS avant de les exposer dans VA pour les gros volumes
- Utiliser les mesures plutôt que les colonnes calculées pour les perfs
- Limiter le nombre d'objets par page (max 8-10) pour la lisibilité
- Tester les rapports sur mobile si la cible inclut des utilisateurs mobiles
- Nommer clairement les mesures et dimensions (noms métier, pas techniques)

5. Machine Learning — SAS Model Studio

SAS Model Studio (Visual Data Mining & ML)

SAS Model Studio est l'interface visuelle pour construire des pipelines ML complets dans SAS Viya. Il orchestre les traitements CAS pour l'apprentissage automatique.

Types de projets

- Régression : prédiction de valeurs continues (ex: scoring de risque crédit)
- Classification binaire : oui/non (ex: défaut, fraude, churn)
- Classification multiclasse
- Clustering : segmentation non supervisée (K-Means, SOM...)
- Prévion de séries temporelles (ARIMA, Prophet, ESM, UCM...)
- Survival analysis : modélisation de la durée jusqu'à un événement
- Text mining : extraction de topics, sentiment, classification de texte

Pipeline AutoML

- Data exploration automatique : statistiques descriptives, distribution, corrélations
- Imputation des valeurs manquantes : mean, median, mode, model-based
- Encodage des variables catégorielles : one-hot, target encoding, WoE
- Réduction de dimension : PCA, feature selection (Gini, IV, corrélation)
- Entraînement simultané de multiples algorithmes
- Comparaison de modèles sur des métriques standardisées
- Champion selection automatique selon le critère choisi
- Génération de score code SAS/Python pour le déploiement

Algorithmes disponibles

- Arbres : Decision Tree, Random Forest, Gradient Boosting (SAS GBDT)
- Linéaires : Logistic Regression, Linear Regression, LASSO, Ridge, ElasticNet
- SVM : Support Vector Machine (classification et régression)
- Neural Networks : MLP, RNN, LSTM, CNN (via SAS DLPy)
- Clustering : K-Means, SOM (Self-Organizing Maps), DBSCAN
- Bayésien : Naive Bayes
- Ensemble : Stacking, Blending
- NLP : Topics (LDA), Text classification, Sentiment
- Computer Vision : ResNet, VGG, InceptionV3 (via DLPy)

6. SAS Model Manager

Rôle et positionnement

SAS Model Manager est le centre de gouvernance des modèles ML de SAS Viya. Il gère le cycle de vie complet des modèles : enregistrement, validation, déploiement, monitoring et retrait.

Cycle de vie d'un modèle

- 1. Développement : création du modèle dans SAS Model Studio, Python, R, ou autre
- 2. Enregistrement : import du modèle dans Model Manager (SAS, Python, R, PMML, ONNX)
- 3. Validation : tests de performance, comparaison avec le champion actuel
- 4. Approbation (workflow) : circuit de validation métier / risque avant production
- 5. Déploiement : publication vers CAS (scoring batch) ou API REST (scoring temps réel)
- 6. Monitoring : suivi des performances en production (dérive, PSI, KS, Gini)
- 7. Retraining ou retrait : décision basée sur les indicateurs de monitoring

Fonctionnalités clés

- Model repository : inventaire centralisé de tous les modèles
- Versioning : historique de toutes les versions d'un même modèle
- Metadata : description, propriétaire, périmètre, date, performances attendues
- Champion/Challenger : comparaison live entre deux modèles en production
- Score code : génération et publication du code de scoring (SAS, Python, DS2)
- Scoring destinations : CAS table, MAS (Micro Analytic Service), SAS Decisioning
- Performance monitoring : métriques de dérive sur données de production
- Alertes : notifications quand les indicateurs passent des seuils
- Audit trail : qui a fait quoi sur quel modèle, quand
- Integration API : pipeline CI/CD via REST APIs

MAS — Micro Analytic Service

- Service de scoring temps réel (latence < 10ms) pour les décisions en ligne
- Exécution du score code SAS DS2 ou Python dans un microservice dédié
- Utilisé pour : scoring fraude temps réel, éligibilité crédit en ligne, recommandation
- Scalable horizontalement via Kubernetes
- Intégré avec SAS Intelligent Decisioning

7. SAS Intelligent Decisioning

Présentation

SAS Intelligent Decisioning (ID) permet d'automatiser des décisions métier complexes en combinant règles métier, modèles ML, et traitements analytiques dans un flux décisionnel orchestré.

Composants d'une décision

- Business rules : règles IF/THEN codées dans un éditeur visuel ou tableur
- Model objects : intégration de modèles ML depuis Model Manager
- Condition nodes : branchements conditionnels dans le flux
- DS2 code nodes : traitement personnalisé en code SAS DS2
- Custom objects : appels à des services externes (REST)
- Treatments : actions déclenchées (offre, décision, message)

Cas d'usage typiques

- Attribution de crédit : combiner score ML + règles métier + règles réglementaires
- Détection de fraude temps réel : score fraude + règles de blocage
- Campagne marketing : éligibilité + ciblage + offre personnalisée
- Tarification dynamique : score risque + règles tarifaires
- Compliance : vérifications KYC/AML automatisées

8. Gestion des données & CASLIBs

CASLIBs

Un CASLIB est une connexion entre CAS et une source de données (filesystem, base de données, cloud). Il définit où CAS peut lire et écrire des tables.

- Types : Path (filesystem SAS), DNFS (NFS distribué), HDFS, S3, ADLS, GCS
- Types bases : JDBC (Oracle, PostgreSQL, MySQL...), Teradata, Snowflake, Redshift
- Scope : session (visible que par l'utilisateur) ou global (partagé)
- Tables en mémoire vs on-disk : SAVE vs PROMOTE pour persister
- Permissions : accès lecture/écriture configurable par groupe

Chargement de données dans CAS

- PROC CASUTIL : chargement de fichiers SAS7BDAT, CSV, Parquet, ORC dans CAS
- Libname CAS statement : connexion CASLIB depuis une session SAS
- DATA step avec output CAS : PROC CAS ou SET/OUTPUT sur une librairie CAS
- REST API : chargement via API pour les pipelines automatisés
- SAS Studio Flows : interface visuelle pour le chargement
- SAS Visual Analytics : import direct depuis l'interface

Data Management & Qualité

- SAS Data Quality (DQ) : standardisation, dédoublonnage, validation
- SAS Data Preparation : interface visuelle de préparation similaire à un ETL
- Lineage des données : traçabilité de la transformation de bout en bout
- PROC FREQ / PROC MEANS / PROC UNIVARIATE : statistiques descriptives standard
- Détection d'outliers : PROC BOXPLOT, PROC ROBUSTREG
- Gestion des formats SAS : formats personnalisés pour l'encodage des valeurs

9. Programmation SAS avancée

CASL — CAS Action Language

CASL est le langage natif pour piloter CAS. Il permet d'exécuter des actions CAS et de manipuler les résultats en code.

- Syntaxe : `proc cas; action.nom / parametre=valeur; run;`
- Ou depuis Python/R : `import swat; conn.action('nom.action', param=valeur)`
- Action sets : groupes d'actions thématiques (table, regression, forest, textMining...)
- Résultats : retournés sous forme de dictionnaire (Python) ou de tableau SAS
- CASL depuis SAS : PROC CAS est l'interface standard pour appeler des actions CAS

Procédures statistiques clés

- PROC REG : régression linéaire classique
- PROC LOGISTIC : régression logistique, odds ratios, courbe ROC
- PROC GLM : modèles linéaires généraux
- PROC MIXED : modèles linéaires mixtes (effets aléatoires)
- PROC PHREG : modèle de Cox (survie)
- PROC ARIMA / PROC ESM / PROC UCM : prévision de séries temporelles
- PROC CLUSTER / PROC FASTCLUS : clustering hiérarchique et K-Means
- PROC PRINCOMP / PROC FACTOR : ACP et analyse factorielle
- PROC FREQ : tableaux de fréquences et tests du chi-deux
- PROC MEANS / PROC SUMMARY : statistiques descriptives
- PROC UNIVARIATE : distributions, tests de normalité
- PROC REPORT / PROC TABULATE : reporting formaté

SAS Viya Python (SWAT)

- SWAT (SAS Wrapper for Analytics Transfer) : librairie Python pour piloter CAS
- `import swat; conn = swat.CAS(host, port, user, password)`
- Exécution d'actions CAS depuis Python : `conn.loadtable()`, `conn.summary()`, etc.
- Résultats retournés comme DataFrames Pandas
- Compatible avec scikit-learn, matplotlib, seaborn pour les workflows hybrides
- Utilisation dans SAS Studio (Python kernel) ou Jupyter Notebook externe

10. Administration SAS Viya

SAS Environment Manager

- Interface web d'administration centralisée pour Viya
- Gestion des utilisateurs, groupes, rôles, permissions
- Monitoring des services : statut des microservices, logs, alertes
- Gestion des CASLIBs et connexions de données
- Quotas et gouvernance des ressources CAS
- Configuration des servers CAS (mémoire, workers, sessions max)

Gestion des identités

- Authentification : LDAP, Active Directory, OAuth 2.0, SAML 2.0
- Groupes SAS : mappage depuis les groupes AD/LDAP
- Rôles : droits fonctionnels par application (VA, Model Manager, Studio...)
- Capabilities : permissions granulaires sur les fonctions de chaque produit
- Row-level security : filtres sur les données selon l'identité de l'utilisateur
- Column masking : masquage de colonnes sensibles selon le groupe

Infrastructure Kubernetes

- Déploiement via Helm charts officiels SAS
- Namespaces Kubernetes : isolation des environnements (dev/test/prod)
- Persistent Volume Claims : stockage persistant pour CAS et les logs
- Resource quotas : CPU/RAM par namespace
- Ingress Controller : exposition des services web (NGINX, HAProxy)
- Secrets Kubernetes : gestion des credentials SAS Vault
- Monitoring : intégration Prometheus/Grafana pour les métriques K8s et CAS

11. Bonnes pratiques

Performance et CAS

- Toujours charger les données dans CAS avant de les analyser — éviter les lectures disque répétées
- Utiliser PROC CASUTIL avec PROMOTE pour partager les tables entre sessions
- Préférer les actions CAS natives aux procédures SAS classiques sur de gros volumes
- Utiliser le partitionnement CAS pour les grandes tables (partition par date, région)
- Libérer les tables CAS après usage (DROPTABLE) pour libérer la mémoire
- Surveiller la mémoire CAS via SAS Environment Manager

Code SAS

- Toujours utiliser des formats pour les variables catégorielles à cardinalité fixe
- Utiliser les macros pour factoriser le code répétitif
- Éviter les boucles DATA step sur des millions de lignes — préférer les traitements BY ou SQL
- Utiliser PROC SQL avec INOBS= pour tester sur un échantillon avant de lancer en prod
- Documenter les macros avec des commentaires en-tête (input, output, auteur, date)
- Stocker les macros réutilisables dans une bibliothèque AUTOCALL partagée

MLOps et gouvernance des modèles

- Enregistrer tous les modèles dans Model Manager — même les modèles en développement
- Documenter chaque modèle : périmètre, données, métriques attendues, règles de retrait
- Mettre en place un workflow d'approbation pour les modèles réglementés
- Monitorer systématiquement les modèles en production : PSI mensuel au minimum
- Conserver le code source et les données d'entraînement pour l'audit réglementaire
- Versionner le score code pour pouvoir rejouer un scoring historique

12. Erreurs fréquentes

Erreurs de programmation SAS

- Oublier le point-virgule à la fin d'une instruction !' SAS attend la suite, pas d'exécution
- PROC SORT sans OUT= !' SAS trie la table en place, destruction des données originales
- Merge sans BY !' jointure cartésienne silencieuse, explosion du volume
- Merge avec BY sans tri préalable !' résultats incorrects (ou erreur si SORTEDBY= absent)
- Variable créée dans un IF sans ELSE !' valeur manquante (. ou ") dans les autres cas
- Confondre = (comparaison) et = (affectation) en DATA step — pas d'erreur, comportement inattendu
- Format manquant sur une variable numérique catégorielle !' mauvaise interprétation
- Dépasser la longueur d'une variable caractère sans ATTRIB/LENGTH !' troncature silencieuse

Erreurs CAS

- Utiliser une librairie de session dans un job de scheduling !' table invisible hors session
- Ne pas promouvoir une table nécessaire par d'autres utilisateurs !' "table not found"
- Charger en CAS une table de plusieurs dizaines de Go sans quota !' crash OOM du cluster
- Modifier une table CAS promue en cours d'utilisation !' résultats incohérents pour les autres
- Oublier de libérer les sessions longues !' saturation du nombre de sessions max
- Ne pas sauvegarder une table CAS avant un redémarrage !' perte des données in-memory

Erreurs Model Manager

- Enregistrer un modèle sans score code !' impossible à déployer pour le scoring
- Oublier de renseigner les variables d'entrée/sortie du modèle !' erreur au déploiement
- Tester le champion/challenger sur les mêmes données d'entraînement !' overfitting non détecté
- Ne pas configurer le monitoring !' dérive non détectée en production
- Déployer un nouveau modèle sans archiver l'ancien !' impossible de rollback
- Ignorer les alertes PSI !' modèle dégradé utilisé en production des semaines sans action

Erreurs d'administration

- CASLIB global accessible à tous par défaut !' exposition de données sensibles
- Négliger les Persistent Volumes !' perte des données CAS lors d'un redémarrage K8s
- Sous-dimensionner la mémoire CAS !' jobs qui tombent par OOM en production
- Ne pas configurer le backup de l'environnement SAS Viya !' risque catastrophique
- Ignorer les logs d'audit !' impossible de retracer une modification réglementaire

13. Éléments de pitch client

Arguments différenciants SAS Viya

- Crédibilité réglementaire : SAS est la référence dans les banques, assurances et gouvernements
- Richesse statistique : 30+ ans de procédures validées scientifiquement
- Conformité et audit : traçabilité de bout en bout des modèles (exigences BCBS 239, SR 11-7)
- Performance CAS : in-memory distribué, facteur x10 à x100 vs SAS 9 traditionnel
- Migration SAS 9 !' Viya : continuité du code existant, pas de réécriture totale
- Sécurité enterprise : row-level, column masking, SSO, audit trail natifs
- SAS Cloud Analytic Services : gestion de la mémoire plus fiable que Spark sur des jobs longs

Scénarios de transformation client

- "Nous avons 15 ans de code SAS 9 impossible à migrer" !' Viya exécute le code SAS 9 natif + ajout de nouvelles capacités
- "Notre équipe MLOps est inexistante" !' Model Manager = gouvernance clés en main, approuvée par les régulateurs
- "Les décisions métier sont codées en dur dans du Java legacy" !' Intelligent Decisioning = règles et modèles séparés, maintenables par les métiers
- "Nos reportings réglementaires prennent des semaines" !' CAS in-memory réduit les temps de calcul de 90%
- "Nos data scientists utilisent Python mais nos modèles doivent être auditable" !' SAS Viya supporte Python natif ET garantit la gouvernance SAS

14. Questions d'entretien technique

Questions fondamentales

- Quelle est la différence entre une table SAS classique (SAS7BDAT) et une table CAS ?
- Expliquez le fonctionnement du CAS : architecture, sessions, tables promues vs non promues.
- Quelle est la différence entre un CASLIB de session et un CASLIB global ?
- Comment chargez-vous un fichier CSV de 50 Go dans CAS efficacement ?
- Expliquez la différence entre PROC CAS et PROC SQL dans SAS Viya.

Questions programmation SAS

- Quelle est la différence entre MERGE et SET dans un DATA step ? Quand utiliser l'un ou l'autre ?
- Comment utilisez-vous les hash objects pour optimiser une jointure dans un DATA step ?
- Expliquez le concept de formats SAS et leur utilité dans un projet de scoring.
- Comment créez-vous et appelez-vous une macro paramétrique en SAS ?
- Quelle est la différence entre %LET et CALL SYMPUT ?

Questions MLOps et Model Manager

- Comment enregistrez-vous un modèle Python dans SAS Model Manager ?
- Expliquez le cycle de vie d'un modèle dans SAS Model Manager.
- Qu'est-ce que le PSI (Population Stability Index) et à partir de quel seuil agissez-vous ?
- Comment déployez-vous un modèle pour le scoring temps réel avec MAS ?
- Comment implémentez-vous un workflow Champion/Challenger dans SAS Viya ?

Questions architecture et performance

- Quels sont les avantages du calcul in-memory CAS vs le traitement disque SAS 9 ?
- Comment optimisez-vous la performance d'un chargement de table de 100 Go dans CAS ?
- Quelle est la différence entre SAS Compute Server et CAS ? Quand utiliser l'un ou l'autre ?
- Comment configurez-vous les partitions CAS pour optimiser les jointures ?
- Comment intégrez-vous SAS Viya avec Azure Data Factory ou Airflow ?

Questions contexte réglementaire (ESG/banque)

- Comment SAS Viya répond-il aux exigences de traçabilité du BCBS 239 ?
- Comment garantisiez-vous la reproductibilité d'un calcul réglementaire dans SAS Viya ?
- Quelles fonctionnalités de SAS Model Manager sont pertinentes pour SR 11-7 / TRIM ?
- Comment implémentez-vous le contrôle d'accès aux données sensibles ESG dans CAS ?
- Comment gérez-vous les versions de code SAS pour les reportings réglementaires annuels ?

